

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;
```

```
struct Person
{
    string first_name;
    string last_name;
    string address;
    string phone;
    int birth_date;
};
```

```
struct Node
{
    Person data;

    Node* left;
    Node* right;
};
```

```
class Person_tree
{
private:

    Node* root;

    int compare(Person a, Person b)
    {
        if (a.last_name < b.last_name)
        {
            return -1;
        }

        if (a.last_name > b.last_name)
        {
            return 1;
        }

        if (a.first_name < b.first_name)
```

```

    {
        return -1;
    }

    if (a.first_name > b.first_name)
    {
        return 1;
    }

    if (a.birth_date < b.birth_date)
    {
        return -1;
    }

    if (a.birth_date > b.birth_date)
    {
        return 1;
    }
}

void add_person(Node*& temp, Person p)
{
    if (temp == NULL)
    {
        temp = new Node;

        temp->data = p;

        temp->left = NULL;
        temp->right = NULL;

        return;
    }

    if (compare(p, temp->data) < 0)
    {
        add_person(temp->left, p);
    }

    else
    {
        add_person(temp->right, p);
    }
}

```

```

int count_In(Node* temp, string last)
{
    if (temp == NULL)
    {
        return 0;
    }

    int count = 0;

    if (temp->data.last_name == last)
    {
        count+=1;

        count += count_In(temp->left, last);
        count += count_In(temp->right, last);
    }

    else if (last < temp->data.last_name)
    {
        count += count_In(temp->left, last);
    }

    else
    {
        count += count_In(temp->right, last);
    }

    return count;
}

```

```

int count_name(Node* temp, string name)
{
    if (temp == NULL)
    {
        return 0;
    }

    int count = 0;

    if (temp->data.first_name == name || temp->data.last_name == name)
    {
        count+=1;
    }
}

```

```

        count += count_name(temp->left, name);
        count += count_name(temp->right, name);

    return count;
}

void find_oldest(Node* temp, Person& oldest)
{
    if (temp == NULL)
    {
        return;
    }

    if (temp->data.birth_date < oldest.birth_date)
    {
        oldest = temp->data;
    }

    find_oldest(temp->left, oldest);
    find_oldest(temp->right, oldest);
}

void save_tree(Node* temp, ofstream& fout)
{
    if (temp == NULL)
    {
        return;
    }

    fout << temp->data.first_name << endl;
    fout << temp->data.last_name << endl;
    fout << temp->data.address << endl;
    fout << temp->data.phone << endl;
    fout << temp->data.birth_date << endl;

    save_tree(temp->left, fout);
    save_tree(temp->right, fout);
}

public:
    Person_tree()
    {
        root = NULL;
    }

```

```

}

void add(Person p)
{
    add_person(root, p);
}

int totalLastName(string last)
{
    return count_ln(root, last);
}

int totalName(string name)
{
    return count_name(root, name);
}

string oldest_person()
{
    if (root == NULL)
    {
        return "Tree empty";
    }

    Person oldest = root->data;

    find_oldest(root, oldest);

    return oldest.first_name + " " + oldest.last_name;
}

void save_to_file(string filename)
{
    ofstream fout;

    fout.open(filename);

    if (fout.fail())
    {
        cout << "File error :(" << endl;
        return;
    }

    save_tree(root, fout);

```

```
        fout.close();
    }
};

int main()
{
    Person_tree tree;

    Person p1;
    p1.first_name = "Beyonce";
    p1.last_name = "Knowles";
    p1.address = "123 Hollywood";
    p1.phone = "111-1111";
    p1.birth_date = 19810904;

    Person p2;
    p2.first_name = "Chris";
    p2.last_name = "Hemsworth";
    p2.address = "456 Hollywood";
    p2.phone = "222-2222";
    p2.birth_date = 19830811;

    Person p3;
    p3.first_name = "Liam";
    p3.last_name = "Hemsworth";
    p3.address = "789 Hollywood";
    p3.phone = "333-3333";
    p3.birth_date = 19900113;

    Person p4;
    p4.first_name = "Zendaya";
    p4.last_name = "Coleman";
    p4.address = "999 Hollywood";
    p4.phone = "444-4444";
    p4.birth_date = 19960901;

    Person p5;
    p5.first_name = "Chris";
    p5.last_name = "Evans";
    p5.address = "101 Hollywood";
    p5.phone = "555-5555";
    p5.birth_date = 19810613;
```

```

tree.add(p1);
tree.add(p2);
tree.add(p3);
tree.add(p4);
tree.add(p5);

cout << "People named Beyonce: " << tree.totalName("Beyonce") << endl;
cout << "People named Liam: " << tree.totalName("Liam") << endl;
cout << "People named Zendaya: " << tree.totalName("Zendaya") << endl;
cout << "People named Chris: " << tree.totalName("Chris") << endl << endl;

cout << "People with last name Hemsworth: " << tree.totalLastName("Hemsworth") << endl;
cout << "People with last name Coleman: " << tree.totalLastName("Coleman") << endl;
cout << "People with last name Knowles: " << tree.totalLastName("Knowles") << endl;
cout << "People with last name Evans: " << tree.totalLastName("Evans") << endl << endl;

cout << "Oldest person: " << tree.oldest_person() << endl;

tree.save_to_file("people.txt");
}

```

## **RESULTS:**

People named Beyonce: 1  
 People named Liam: 1  
 People named Zendaya: 1  
 People named Chris: 2

People with last name Hemsworth: 2  
 People with last name Coleman: 1  
 People with last name Knowles: 1  
 People with last name Evans: 1

Oldest person: Chris Evans